

Petalized Compute Allocation Model

Resource Partitioning as Botanical Abstraction

VSEPR-SIM Research Platform

Section §P — Theoretical Foundation

Version 3.0.0 — First Principles Draft

March 29, 2026

Contents

§1 Motivation: The Botanical Metaphor for Computation

Modern compute environments expose resources as anonymous pools: a count of cores, a quantity of memory, a fractional share of accelerator time. While technically sufficient, this flat representation obscures the *structure* of resource availability and the *dependency relationships* that connect visible computation to its supporting infrastructure.

The **Petalized Compute Allocation Model** introduces a botanical metaphor that maps resource partitioning onto the anatomy of a flower. The metaphor is not decorative; it encodes a specific hierarchy:

- The **bloom** is the exposed computational surface of the machine.
- Each **petal** is a schedulable compute partition.
- The **roots** and **stem** represent the non-visible support systems required to sustain execution: power delivery, thermal stability, storage movement, network access, and system coordination.

The metaphor thus captures both the *beauty of variation* (heterogeneous resource profiles across partitions) and the *physical dependency* of computation on deeper infrastructural support.

§2 Formal Flower Decomposition

Definition 2.1 (Flower). A **flower** F is a quadruple

$$F = \{R, S, C, P\} \quad (1)$$

where each component represents a distinct functional layer of the compute system.

R — Root Systems (Support Layer)

Power delivery, cooling, storage, network ingress, persistence. The root systems are the non-visible infrastructure upon which all computation ultimately depends. A root failure collapses the entire flower.

S — Stem (Transport and Control Backbone)

Scheduler path, process routing, kernel coordination. The stem connects the exposed bloom to the root systems, providing the transport and arbitration layer through which resource requests propagate.

C — Core (Orchestration Center)

Allocator, supervisor, node manager, system controller. The core is the decision point: it determines how resources are distributed across petals and how competing demands are resolved.

P — Petals (Exposed Compute Partitions)

The set of schedulable compute partitions:

$$P = \{p_1, p_2, \dots, p_n\} \quad (2)$$

Each petal is a bounded, explicitly described region of machine capability visible to user-facing workloads.

§3 Petal Structure

Definition 3.1 (Petal). Each petal p_i is defined as

$$p_i = \{M_i, C_i, G_i, D_i\} \quad (3)$$

where:

- M_i — memory assigned to petal i (e.g. in GB),
- C_i — CPU cores assigned to petal i ,
- G_i — GPU fraction or accelerator share assigned to petal i ($0 \leq G_i \leq 1$),
- D_i — number of internal divisions (sub-partitions) available within petal i .

Remark 3.2 (Sub-division). The internal divisions D_i allow tasks to be distributed at finer granularity within the same visible compute region. A petal with $D_i = 4$ can host up to four independently scheduled work units, each receiving an equal share of M_i , C_i , and G_i unless explicitly overridden by the orchestration core.

3.1 Concrete Example

Consider a single node represented as a flower whose petals each expose:

$$p_i = \{25.6 \text{ GB}, 6 \text{ cores}, 0.30 \text{ GPU}, 4\} \quad (4)$$

This states that petal i provides 25.6 GB of memory, 6 CPU cores, 30% of a GPU allocation, and 4 sub-divisions for internal scheduling.

§4 Bloom Capacity

Definition 4.1 (Bloom Capacity). The **total exposed bloom capacity** B of a flower with n petals is

$$B = \sum_{i=1}^n p_i \quad (5)$$

interpreted component-wise:

$$B = \left(\sum_{i=1}^n M_i, \sum_{i=1}^n C_i, \sum_{i=1}^n G_i, \sum_{i=1}^n D_i \right) \quad (6)$$

For a flower with $n = 4$ identical petals as in the example above:

$$B = (102.4 \text{ GB}, 24 \text{ cores}, 1.20 \text{ GPU}, 16) \quad (7)$$

Note that $\sum G_i > 1$ is valid when a node contains multiple physical accelerators or when GPU fractions are expressed relative to a single device among several.

§5 Heterogeneous Petal Configurations

The petalized model is particularly suitable for heterogeneous or mixed-load systems. Petals need not be uniform:

Petal	Memory (GB)	CPU Cores	GPU Share	Divisions
p_1 (CPU-dense)	16.0	12	0.00	6
p_2 (Memory-heavy)	64.0	4	0.00	2
p_3 (Accelerated)	8.0	2	0.50	1
p_4 (Balanced)	25.6	6	0.30	4

Color, placement, and visual encoding may then represent:

- node role (compute, storage, accelerator),
- priority class (interactive, batch, preemptible),
- thermal state (nominal, throttled, critical),
- resource pressure (idle, loaded, saturated).

A petal is therefore both a **compute unit** and a **visual contract**: it tells the user what resources exist, how they are divided, and where tasks are currently placed.

§6 Scheduling Interpretation

The flower decomposition provides a natural mapping to scheduling abstractions:

1. **Petal selection** — the scheduler selects a petal p_i whose resource profile matches the task requirements.
2. **Division assignment** — within the selected petal, the task is assigned to one of D_i available sub-partitions.
3. **Resource binding** — the task receives its share of M_i , C_i , and G_i proportional to the division count.
4. **Bloom monitoring** — the orchestration core C monitors aggregate bloom utilization and may rebalance across petals.

This hierarchy maps cleanly onto existing container orchestration, NUMA-aware scheduling, and GPU partitioning (e.g. MIG, MPS) paradigms while providing a human-readable structural overlay.

§7 Infrastructure Dependencies

The root and stem layers encode a critical design principle: *visible computation depends on invisible infrastructure*. A flower whose roots are compromised (e.g. insufficient cooling, failing storage) cannot sustain its bloom regardless of how well petals are configured.

7.1 Root System Components

- **Power delivery** — wattage budget, PSU redundancy, UPS state.
- **Cooling** — ambient temperature, fan RPM, thermal throttle thresholds.
- **Storage** — IOPS capacity, read/write bandwidth, persistence guarantees.
- **Network ingress** — link bandwidth, latency, packet loss rate.

7.2 Stem Components

- **Scheduler path** — queue depth, dispatch latency, fairness policy.
- **Process routing** — affinity masks, NUMA topology, interrupt steering.
- **Kernel coordination** — syscall overhead, context-switch cost, lock contention.

§8 Thermal Monitoring Integration

The root layer naturally incorporates thermal monitoring as a first-class concern. Temperature readings from hardware sensors can be converted to a consistent Celsius representation:

$$T_C = \frac{T_{\text{raw}}}{10} - 273.15 \quad (8)$$

where T_{raw} is the value reported by firmware interfaces such as `MsAcpi_ThermalZoneTemperature` on Windows systems. Thermal state feeds back into petal scheduling: a thermally stressed

root may force the orchestration core to reduce bloom capacity by disabling or throttling individual petals.

§9 GPU Telemetry as Petal Metadata

For accelerator-bearing petals, runtime telemetry enriches the petal description with dynamic state:

- **GPU utilization** u_{gpu} — fraction of active SMs (0–100%).
- **VRAM usage** m_{gpu} — memory currently allocated (in MB or GB).

These values are cached to avoid per-prompt overhead (e.g. querying `nvidia-smi` on every render cycle). A cache lifetime of ~ 2 s balances freshness against launch cost.

Color encoding of GPU state within the petal visualization follows the convention:

Metric	Threshold	Color
$u_{\text{gpu}} \geq 80\%$	High load	Red (ANSI 196)
$u_{\text{gpu}} \geq 50\%$	Medium load	Orange (ANSI 208)
$u_{\text{gpu}} < 50\%$	Low load	Green (ANSI 118)
m_{gpu}	Any	Light cyan (ANSI 159)

§10 Relationship to the VSEPR-SIM Platform

Within VSEPR-SIM, the petalized compute allocation model serves as the conceptual bridge between the atomistic simulation kernel and the physical hardware that executes it. The mapping is:

- **Atomistic structure generation** tasks are assigned to petals based on memory and core requirements.
- **Visualization rendering** tasks may require accelerator-bearing petals with nonzero G_i .
- **Reporting and export** tasks are lightweight and may occupy minimal petals or share divisions within a larger petal.
- **Discovery workflows** involving parameter sweeps distribute candidate evaluations across the full bloom B .

This ensures that resource allocation is explicit, inspectable, and deterministic — consistent with the anti-black-box philosophy of the platform.

§11 Summary

The Petalized Compute Allocation Model provides:

1. A structured decomposition of compute resources into a four-layer botanical hierarchy: roots, stem, core, and petals.
2. A formal definition of each petal as an explicit tuple of memory, CPU, GPU, and sub-division capacity.
3. A component-wise bloom capacity metric that aggregates across all petals.
4. Support for heterogeneous configurations where petals vary in resource profile and scheduling role.
5. Natural integration with thermal monitoring, GPU telemetry, and infrastructure health as feedback signals.
6. A human-readable visual contract that makes resource state inspectable at every level.

The model is both a **scheduling abstraction** and an **interface abstraction**: it organizes computation into explicit, auditable partitions while providing a visual language for communicating system state to the researcher.